



Zarządzanie procesem produkcji oprogramowania

Lab I

Mgr inż. Marek Andryszczyk

Wprowadzenie

Zarządzanie produkcją oprogramowania to kluczowy aspekt w dużych i małych projektach IT. Błędy na tym etapie mogą prowadzić do znacznych kosztów, opóźnień i niepowodzeń projektów

Celem przedmiotu jest zrozumienie kluczowych pojęć związanych z produkcją oprogramowania, w tym znajomość metodologii Agile i Waterfall, technik zarządzania zespołami oraz narzędzi wspierających ciągłe dostarczanie produktu

Wprowadzenie

„Proces wytwarzania oprogramowania to sekwencja działań, które prowadzą do stworzenia oprogramowania o określonych funkcjonalnościach i jakości w ściśle określonym terminie.”

Proces produkcji oprogramowania

1. Planowanie - etap ten polega na określeniu wymagań użytkownika, stworzeniu harmonogramu projektu, budżetu i wyborze odpowiednich narzędzi i metodologii.
2. Analiza - w tym etapie określa się wymagania funkcjonalne i нефункционалне dla tworzonego oprogramowania. Tworzone są specyfikacje funkcjonalne, dokumenty wymagań oraz analiza kosztów i ryzyk.
3. Projektowanie - etap ten skupia się na stworzeniu architektury oprogramowania, projektowaniu interfejsów użytkownika, modelowaniu bazy danych, tworzeniu diagramów i schematów.
4. Implementacja - na tym etapie programiści piszą kod, testują go i integrują z resztą systemu. Wykonywane są testy jednostkowe, integracyjne i systemowe.
5. Testowanie - etap ten skupia się na sprawdzeniu jakości oprogramowania, zgodności z wymaganiami użytkownika i sprawdzenie poprawności działania. Testowane są zarówno funkcjonalności jak i нефункционалności systemu.
6. Wdrożenie - w tym etapie oprogramowanie jest udostępniane użytkownikom końcowym. Wdrażane są także rozwiązania związane z obsługą, konfiguracją oraz utrzymaniem systemu.
7. Utrzymanie - etap ten obejmuje zarządzanie, aktualizację i konserwację systemu po jego wdrożeniu. Wykonywane są także testy sprawdzające poprawność działania oraz monitorowanie systemu.

Role w zespole IT - Project Manager

- **Główny cel:** Zarządzanie dostarczeniem projektu w ramach ustalonego czasu, zakresu i budżetu.
 - Tworzenie planów projektowych, harmonogramów i harmonogramów realizacji.
 - Koordynacja zasobów zespołu i przydzielanie zadań.
 - Zarządzanie ryzykiem, budżetem i harmonogramami.
 - Komunikacja ze wszystkimi interesariuszami, informowanie o postępach i rozwiązywanie problemów.
 - Zapewnienie, że projekt spełnia standardy jakości i oczekiwania klienta.
 - Wymagane umiejętności:
 - Przywództwo i zarządzanie zespołem.
 - Zarządzanie ryzykiem i rozwiązywanie problemów.
 - Planowanie, harmonogramowanie oraz zarządzanie budżetem.

Role w zespole IT - Product Manager

- **Główny cel:** Zarządzanie wizją i strategią produktu.
- **Odpowiedzialność:**
 - Definiowanie strategii i mapy drogowej produktu.
 - Zbieranie wymagań klientów oraz badania rynku.
 - Priorytetyzacja funkcji i zarządzanie backlogiem produktu.
 - Ścisła współpraca z zespołem programistycznym, aby produkt spełniał potrzeby użytkowników.
 - Dopasowanie celów produktu do celów biznesowych firmy.
- **Wymagane umiejętności:**
 - Myślenie strategiczne i analiza rynku.
 - Doskonała komunikacja oraz umiejętność ustalania priorytetów.
 - Głębokie zrozumienie potrzeb użytkowników i celów biznesowych

Role w zespole IT - Business Analyst

- **Główny cel:** Zbieranie wymagań oraz tłumaczenie potrzeb biznesowych na specyfikacje techniczne.
- **Odpowiedzialność:**
 - Analiza, dokumentowanie i zbieranie wymagań biznesowych.
 - Współpraca z interesariuszami w celu sprecyzowania celów projektu.
 - Przekazywanie wymagań do zespołu deweloperskiego i projektowego w sposób jasny i zrozumiały.
 - Definiowanie zakresu projektu oraz propozycji rozwiązań.
 - Udział w fazie testów poprzez weryfikację, czy rozwiązania spełniają potrzeby biznesowe .
- **Wymagane umiejętności:**
 - Silne zdolności analityczne i rozwiązywanie problemów.
 - Umiejętność efektywnej komunikacji oraz dokumentowania wymagań.
 - Umiejętność łączenia zespołów biznesowych i technologicznych

Role w zespole IT - Software Developer / Software Engineer

- **Główny cel:** Pisanie, testowanie i utrzymywanie kodu aplikacji.
- **Odpowiedzialność:**
 - Tworzenie funkcji na podstawie specyfikacji technicznych i wymagań biznesowych.
 - Pisanie czystego, wydajnego kodu.
 - Współpraca z innymi programistami w zakresie architektury oprogramowania.
 - Przeprowadzanie testów jednostkowych i integracyjnych.
 - Utrzymywanie istniejącego oprogramowania poprzez poprawki błędów i aktualizacje.
- **Wymagane umiejętności:**
 - Znajomość języków programowania (np. Python, Java, C#, JavaScript).
 - Umiejętność rozwiązywania problemów i myślenie analityczne.
 - Zrozumienie zasad programowania obiektowego i wzorców projektowych.

Role w zespole IT - Tester

- **Główny cel:** Zapewnienie jakości oprogramowania poprzez testowanie i weryfikację.
- **Odpowiedzialność:**
 - Tworzenie planów testowych i przypadków testowych.
 - Przeprowadzanie testów funkcjonalnych, integracyjnych i regresyjnych.
 - Zgłaszanie błędów oraz współpraca z programistami w celu ich naprawy.
 - Automatyzacja testów (jeśli odpowiednie narzędzia są stosowane).
 - Weryfikowanie, czy oprogramowanie spełnia wymagania biznesowe i techniczne
- **Wymagane umiejętności:**
 - Znajomość narzędzi do testowania.
 - Dokładność i umiejętność krytycznego myślenia.
 - Wiedza na temat różnych metod testowania (testowanie manualne, automatyczne)

Role w zespole IT - Architekt Systemu

- **Główny cel:** Projektowanie i nadzorowanie architektury technicznej systemu.
- **Odpowiedzialność:**
 - Definiowanie ogólnej struktury i technologii używanych w projekcie.
 - Zapewnienie, że architektura systemu spełnia zarówno wymagania biznesowe, jak i techniczne.
 - Nadzór nad wyborem technologii oraz integracją różnych komponentów systemu.
 - Rozwiązywanie problemów architektonicznych i technicznych w trakcie realizacji projektu.
 - Współpraca z zespołami programistycznymi i operacyjnymi.
- **Wymagane umiejętności:**
 - Znajomość wielu technologii (frontend, backend, bazy danych, chmura).
 - Umiejętność projektowania złożonych systemów informatycznych.
 - Znajomość wzorców architektonicznych i zasad projektowania skalowalnych rozwiązań.

Role w zespole IT - Specjalista ds. bezpieczeństwa

- **Główny cel:** Ochrona systemu i danych przed zagrożeniami.
- **Odpowiedzialność:**
 - Przeprowadzanie audytów bezpieczeństwa i oceny ryzyka.
 - Wdrażanie środków bezpieczeństwa w aplikacjach i systemach IT.
 - Monitorowanie i zapobieganie atakom (np. DDoS, phishing, malware).
 - Tworzenie procedur na wypadek incydentów bezpieczeństwa.
 - Współpraca z zespołem w celu wdrażania najlepszych praktyk w zakresie bezpieczeństwa.
- **Wymagane umiejętności:**
 - Znajomość kryptografii, protokołów bezpieczeństwa i zabezpieczeń sieciowych.
 - Zrozumienie zagrożeń cyberbezpieczeństwa i technik zapobiegania im.
 - Znajomość narzędzi monitorowania i ochrony systemów.

Metodologia projektowania oprogramowania

- Waterfall: (model kaskadowy) to tradycyjna metodologia stosowana w produkcji oprogramowania, oparta na liniowym i sekwencyjnym podejściu.
 - **Zbieranie wymagań** – na tym etapie zbierane są wszystkie wymagania funkcjonalne i нефункционалне dotyczące oprogramowania. Klient określa swoje potrzeby, a zespół tworzy dokumentację, która będzie podstawą dalszych działań.
 - **Projektowanie systemu** – na podstawie wymagań tworzy się szczegółowy projekt systemu. Projekt ten obejmuje zarówno architekturę systemu, jak i szczegółowe schematy dla poszczególnych modułów.
 - **Implementacja** – na tym etapie programiści zaczynają kodowanie według założeń projektu. Każdy moduł jest tworzony oddzielnie, a następnie integrowany w całość.
 - **Testowanie** – po zakończeniu implementacji system przechodzi przez fazę testów, które mają na celu wykrycie błędów i upewnienie się, że system działa zgodnie z założeniami.
 - **Wdrożenie** – gdy oprogramowanie przejdzie testy, następuje jego wdrożenie na środowisku produkcyjnym, czyli przekazanie klientowi do użytkowania.
 - **Utrzymanie** – po wdrożeniu oprogramowania często konieczne jest wprowadzenie poprawek, aktualizacji lub nowych funkcji. Etap utrzymania obejmuje zarówno korektę błędów, jak i wprowadzanie nowych wymagań.

Metodologia projektowania oprogramowania

- **Kluczowe cechy modelu Waterfall:**
- **Sekwencyjność** – każdy etap musi być zakończony przed rozpoczęciem następnego.
- **Brak iteracyjności** – model nie przewiduje powrotu do poprzednich etapów, co oznacza, że zmiany są trudne do wprowadzenia na późniejszych etapach.
- **Dokumentacja** – duży nacisk kładziony jest na dokumentację na każdym etapie.

Waterfall jest metodą stosowaną w projektach, gdzie wymagania są jasno określone od początku, a zmiany są minimalne. Jednak w dzisiejszych dynamicznych środowiskach, bardziej adaptacyjne modele, jak Agile, często zastępują Waterfall.

Metodologia projektowania oprogramowania

- Metodologia Agile to podejście do wytwarzania oprogramowania, które koncentruje się na elastyczności, współpracy z klientem i szybkim dostarczaniu wartościowych funkcji. .
 - **Iteracyjny rozwój** – Agile opiera się na krótkich cyklach rozwojowych zwanych sprintami (zwykle 1-4 tygodni). Każdy sprint kończy się dostarczeniem działającego fragmentu oprogramowania, co pozwala na częste dostarczanie wartości dla użytkownika.
 - **Współpraca z klientem** – W Agile klient jest aktywnie zaangażowany w proces tworzenia oprogramowania. Regularne spotkania i prezentacje wyników pracy (demonstracje) pozwalają na bieżąco dostosowywać produkt do potrzeb klienta.
 - **Zmieniające się wymagania** – Agile zakłada, że wymagania mogą się zmieniać w trakcie projektu. Zamiast sztywnego trzymania się początkowych ustaleń, zespół reaguje na zmiany i dostosowuje oprogramowanie, aby jak najlepiej odpowiadało potrzebom użytkowników.
 - **Samozarządzające się zespoły** – Zespoły Agile mają dużą autonomię i są odpowiedzialne za organizację własnej pracy. Współpraca między programistami, testerami, analitykami i innymi członkami zespołu jest kluczowa.
 - **Częste dostarczanie** – Celem Agile jest szybkie dostarczanie działających części oprogramowania. Zamiast czekać na zakończenie całego projektu, zespoły regularnie oddają klientowi gotowe funkcje, które można od razu wdrożyć.
 - **Stała poprawa** – Po każdym sprincie zespoły Agile przeprowadzają retrospektywy, aby zidentyfikować, co można poprawić w kolejnych iteracjach. Jest to kluczowy element ciągłego doskonalenia.

Metodologia projektowania oprogramowania

- **Kluczowe cechy modelu Agile:**
- Elastyczność – Agile umożliwia szybkie dostosowanie się do zmian w wymaganiach i priorytetach.
- Zaangażowanie klienta – Klient bierze aktywny udział w procesie tworzenia oprogramowania.
- Krótkie cykle dostarczania – Oprogramowanie jest dostarczane w małych, regularnych porcjach.

Agile zyskało popularność w szybko zmieniających się środowiskach, gdzie szybka reakcja na potrzeby klienta i dynamiczne zmiany są kluczowe dla sukcesu projektu.

Metody szacowania pracochłonności i harmonogramy

- Eksperckie metody szacowania
 - Analogowe
 - Widełkowe
- Metody bazujące na podziale na zadania
- Metody probabilistyczne
- Zwinne metody szacowania

Metody szacowania pracochłonności i harmonogramy

- Gantt Chart
- Diagram sieciowy
- Metoda ścieżki krytycznej
- Zwinne metody szacowania

Narzędzia do tworzenia harmonogramów i szacowania (**Jira, Trello, Microsoft Project** czy **Asana**)